

Efficient Data Storage Systems

Sistemas de Armazenamento de Dados Eficientes

Introduction

University of Minho
2025-2026



The Landscape

Advanced Computing

- **Advanced Computing** is shaping the speed and efficiency of how scientists and engineers handle complex and large computational problems
 - **Scientific workloads** for physics, earthquake simulations, drug discovery, ...
 - **Product design and optimization** for aerospace, automotive, energy, and cybersecurity industries
 - **Big Data** analysis and training of large and complex **Artificial Intelligence** models (e.g., deep learning, large language models)
- To run efficiently, these workloads require **specialized hardware** resources and **scale!**

The Landscape

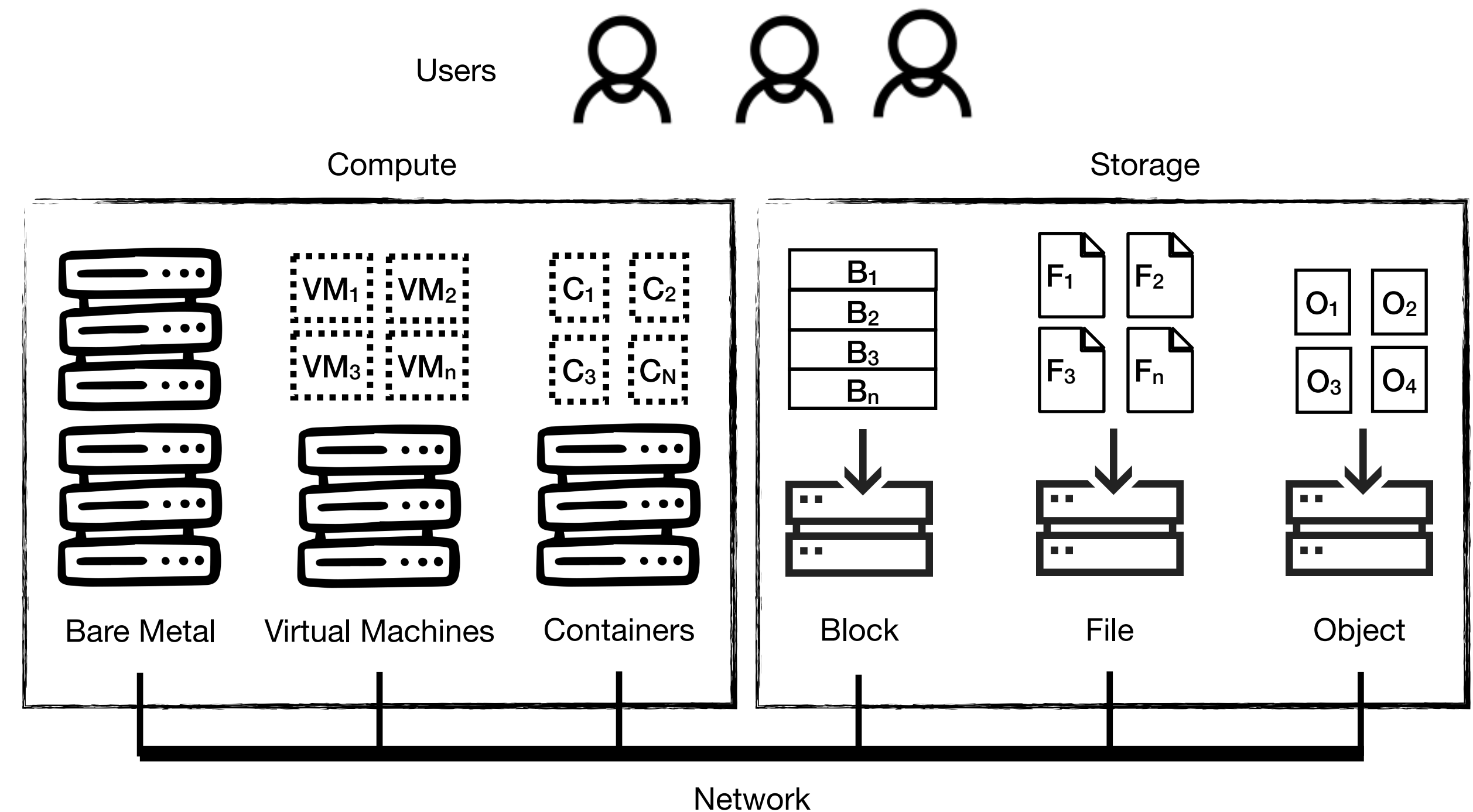
Infrastructures

- The computational and storage demands of current workloads led to increasingly **large and complex HPC infrastructures**
 - **Deucalion:** 1 632 ARM nodes and a 10.6 PetaByte (PB) storage system
 - **Vista:** 600 Grace Hopper nodes (~35 000\$ per chip) and a 30 PB storage system
 - **ABCI 3.0:** 6 128 NVIDIA H200 (~30 000\$ per GPU) and a 75 PB storage system
 - **Fugaku:** 158 972 ARM servers and a 150 PB storage system
- The relevance of these workloads also led many **Cloud Computing** vendors to offer specialized HPC services
- And (one) potential future direction is combining **Quantum Computing** with HPC infrastructures

The Landscape

Traditional Cloud

- Relies on the **virtualization** of storage and computing resources
- Offers several computing and storage interfaces/services
- Bare-metal commodity¹ **resources shared** across users
 - Reduced hardware costs
- Originally planned for hosting services at scale and high availability



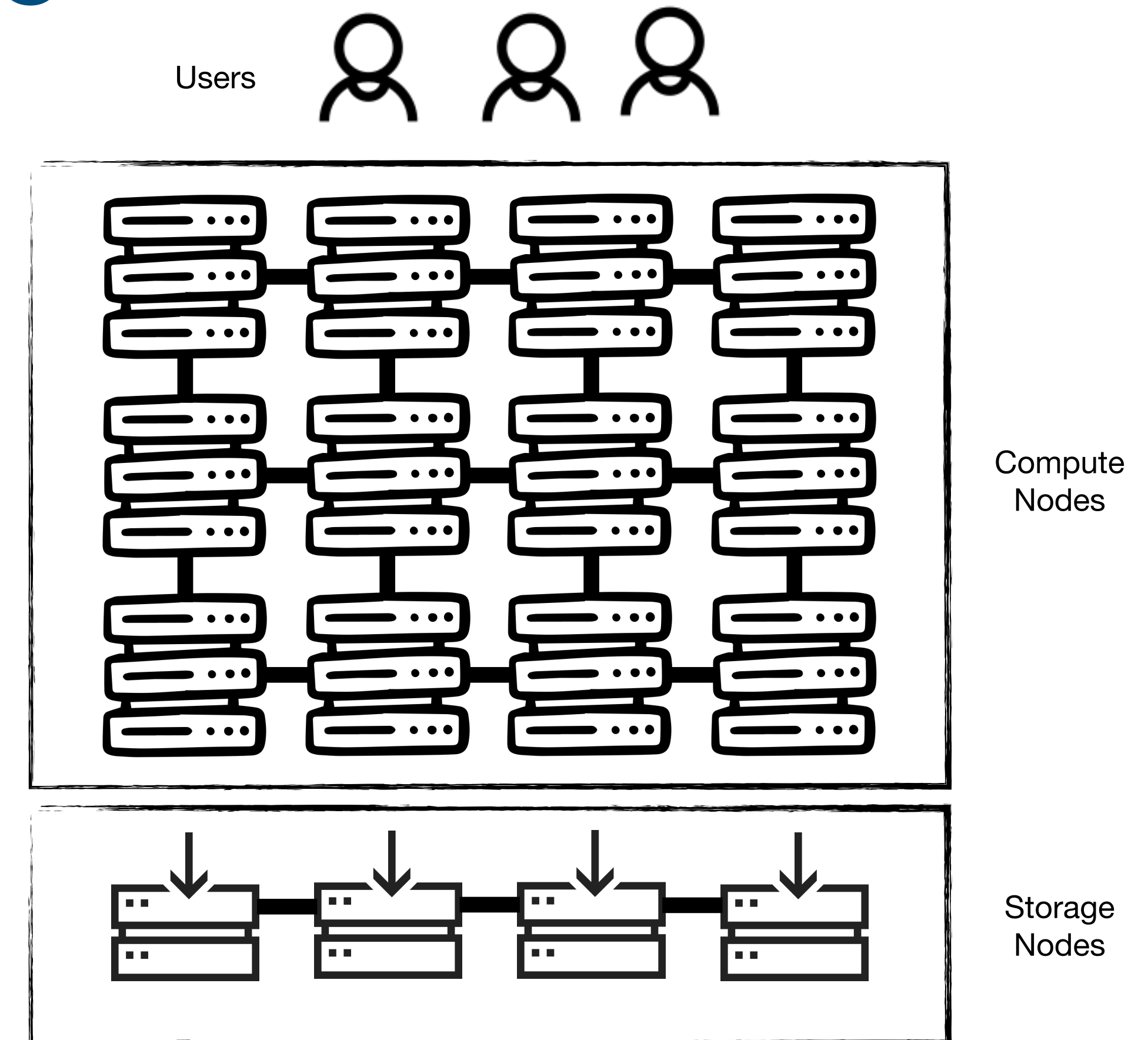
A simplified Cloud infrastructure overview
(based on the OpenStack architecture)

¹ affordable, standardized, and mass-produced computing hardware system

The Landscape

Traditional HPC

- **Bare-metal access** to compute nodes
 - Interactive SSH connection to nodes
 - Batch scripts
- **High-end and specialized** computing, network, and storage **hardware**
 - *E.g.*, CPUs, GPUs, Infiniband, ...
- Designed for running compute-intensive applications (and I/O intensive¹)



A simplified HPC infrastructure overview

¹ with some challenges, as we will see during the class

Enters the Storage Stack

HPC Infrastructure

● Compute nodes

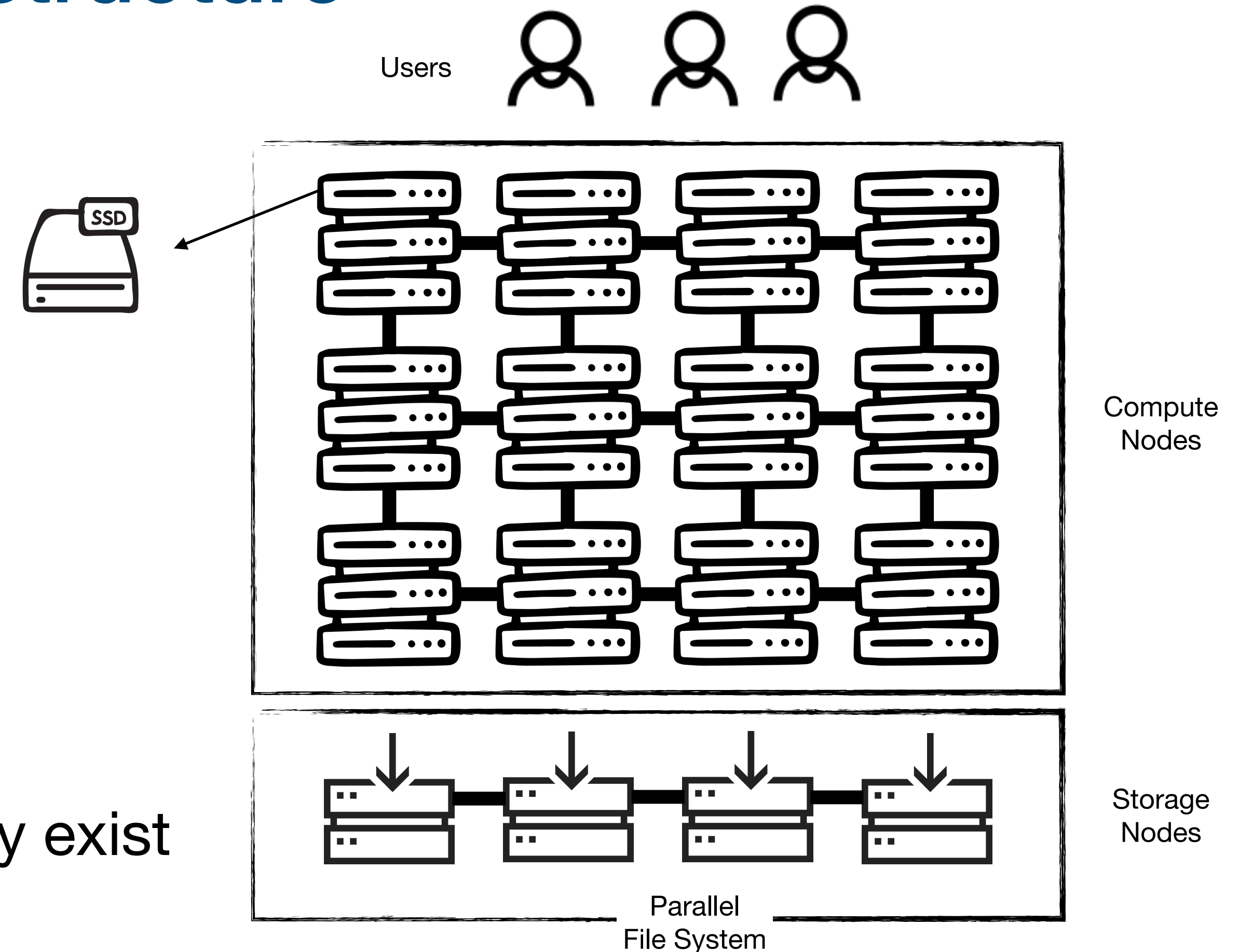
- Local disk(s) per compute node (e.g., HDD, SATA SSD, NVMe SSD)

● Parallel File System (PFS)

- Shared across all compute nodes
- POSIX API
- Composed of a set of storage nodes (*i.e.*, servers dedicated to storing data)

● Other storage backends and interfaces may exist

- E.g., NFS for user homes, Object store, ...

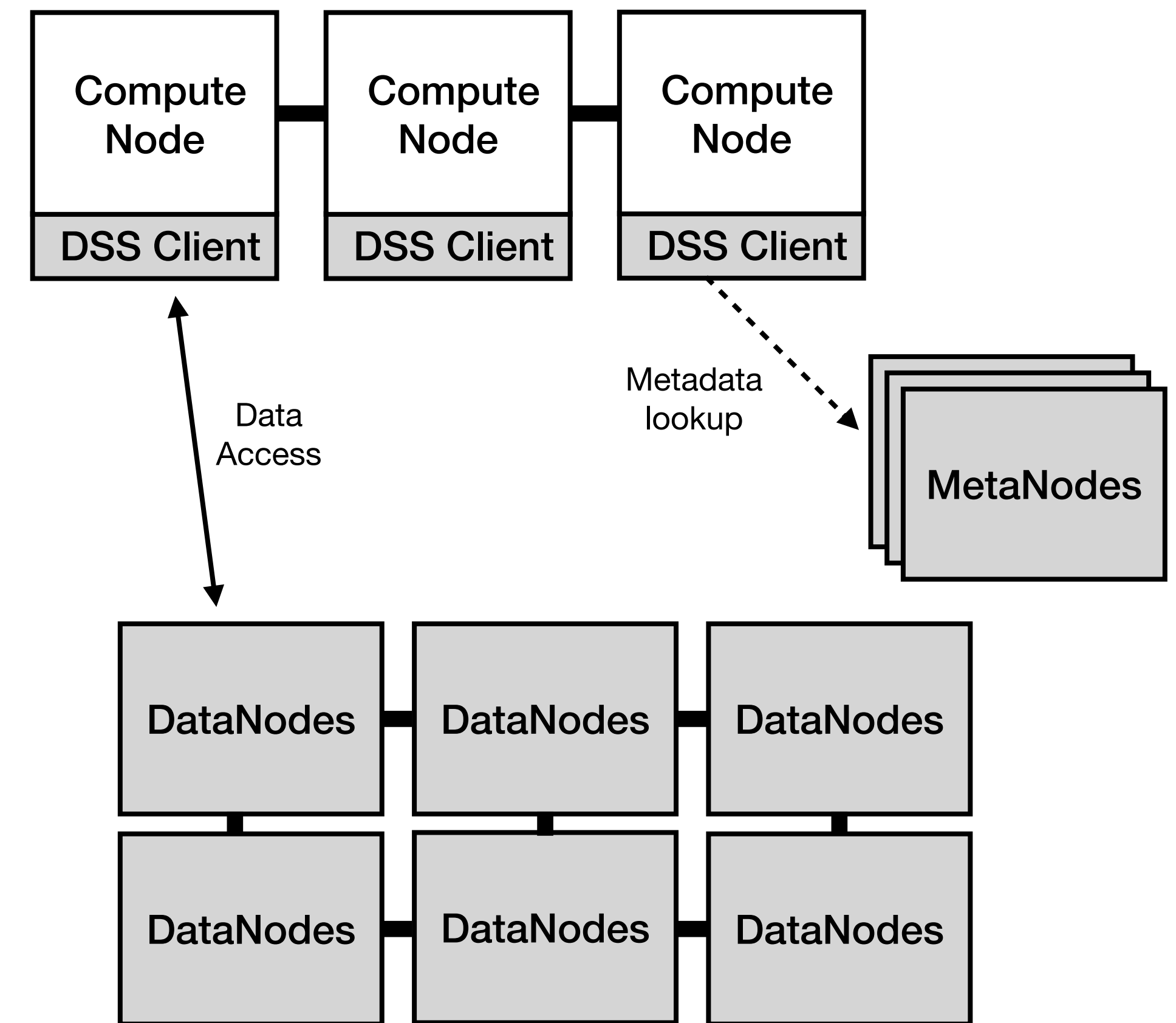


A simplified HPC infrastructure overview

Enters the Storage Stack

Distributed Storage Systems

- Tens to hundreds of nodes storing data
 - Examples: HDFS, Ceph, Lustre, GPFS¹
- Data distributed across **DataNodes** (replicated/sharded)
 - Data requests done directly to DataNodes
- Metadata managed by **MetaNodes** (e.g., location of files, permissions)
 - MetaNodes are used to get data's location

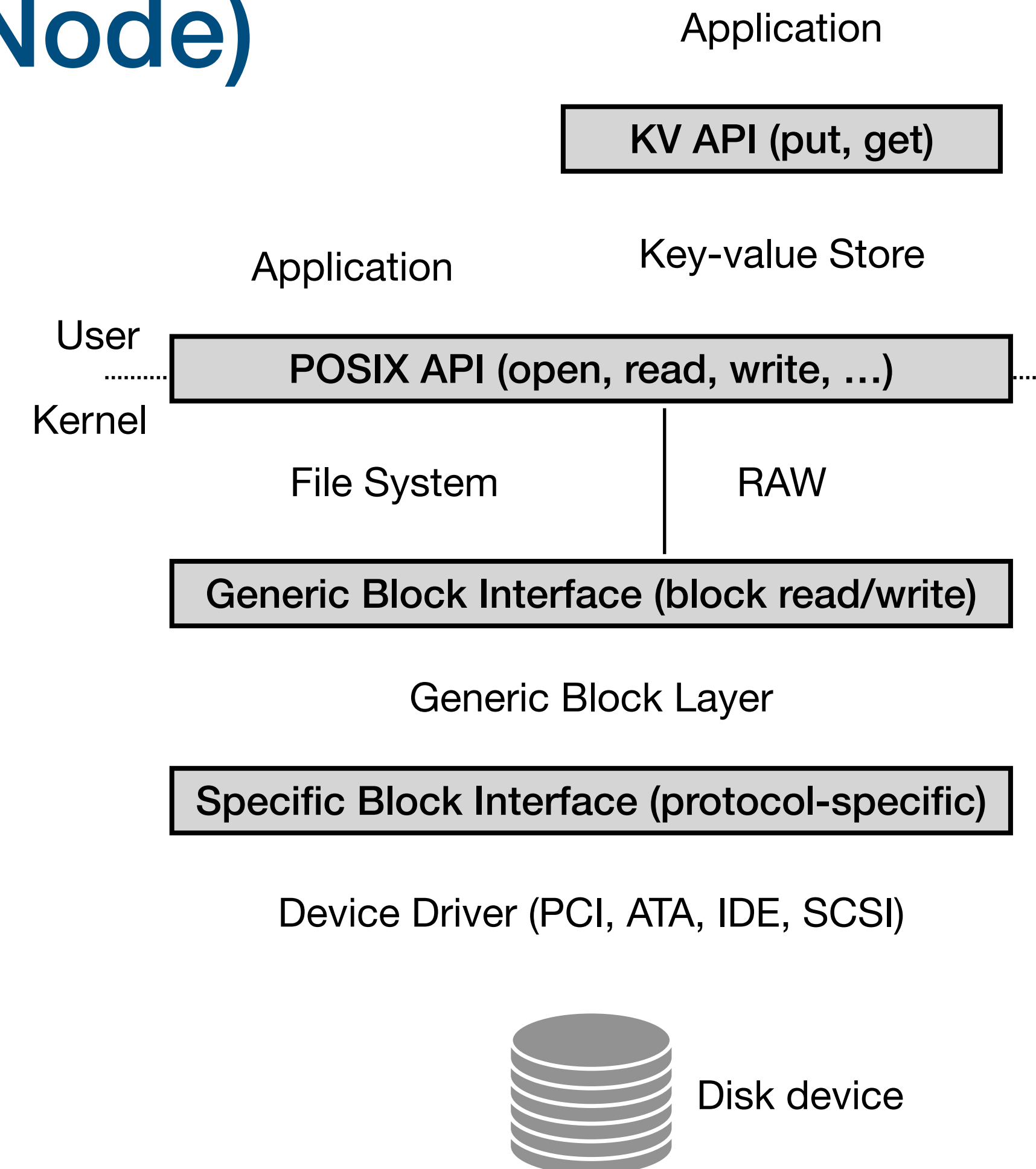


A simplified Distributed Storage System (DSS)

¹ **Disclaimer:** not all these systems follow the design on the right (we will discuss this later on)

Enters the Storage Stack

Local Node (e.g., a DataNode)



● Block Device

- Data is managed as a set of blocks (the closest abstraction to the disk)
- Examples: Linux block device

● File System (FS)

- Data is managed as a hierarchy of directories and files
- Examples: Ext4, ZFS

● Key-value Store

- Data is managed as key-value pairs (i.e., the key is a unique identifier, and the value is the content)
- Examples: LevelDB, RocksDB

A simplified local storage stack

The Storage I/O Challenges

Perspective of System Developers and Administrators

- ◎ How can one design storage systems that meet the scale, performance, dependability, and security requirements of large-scale infrastructure?
- ◎ How can one efficiently configure and tune shared storage resources (e.g., a PFS) to handle **I/O operations from thousands of user jobs**?
- ◎ What **storage optimizations** should be applied along the I/O path (i.e., from the client's application to the node(s) to the local disk(s))?

The Storage I/O Challenges

Application Developers' perspective

- What **storage interfaces** are the best for my application?
- What **I/O patterns** can improve the performance of my application?
 - *E.g.*, sequential vs random I/O, large vs small file I/O
- What **I/O optimizations** can help improve the performance of my application?

Why Should I Do This Course?

- ◎ Storage systems are critical for the efficiency and reliability of data-centric applications
- ◎ The incorrect design, deployment, use, and tuning of storage systems may lead to
 - Inefficient programs (bad performance, poor resource usage)
 - Incorrect programs (bugs, crashes, data corruption)
- ◎ To correctly use these systems and APIs, one must:
 - Build and try them - **practical classes!**
 - Understand their internals - **theoretical classes!**
- ◎ And finally, because **Storage Systems are Fun!**
 - You will study algorithms, data structures, and optimizations applicable across different contexts (AI pipelines, databases, web services, apps, ...).
 - Knowing how to do efficient storage I/O is useful for several careers
 - Cloud/HPC administrators, system developers, application developers, data scientists, ...

Roadmap

Bottom-up approach

Theory

- Storage Stack and Interfaces
 - Storage Devices
 - Block Devices, File Systems, and Key-Value Stores
- Storage Optimizations
 - Caching and Tiering
 - Data Reduction and Security
- Distributed Storage and Optimizations
 - Remote and Distributed Storage Systems
 - Scalability, Consistency and Reliability
- Storage Benchmarking
 - Storage Benchmarks and Traces

Practice

- Warmup
 - Revision of Threading and Data Structures
- Local Storage and Optimizations
 - The FUSE Framework
 - Key-value Stores
 - Caching and Tiering
 - Data Reduction and Security
- Remote and Distributed Storage
 - Remote and Distributed File System
 - Data Replication
- Storage Benchmarking
 - FIO and Filebench benchmarks

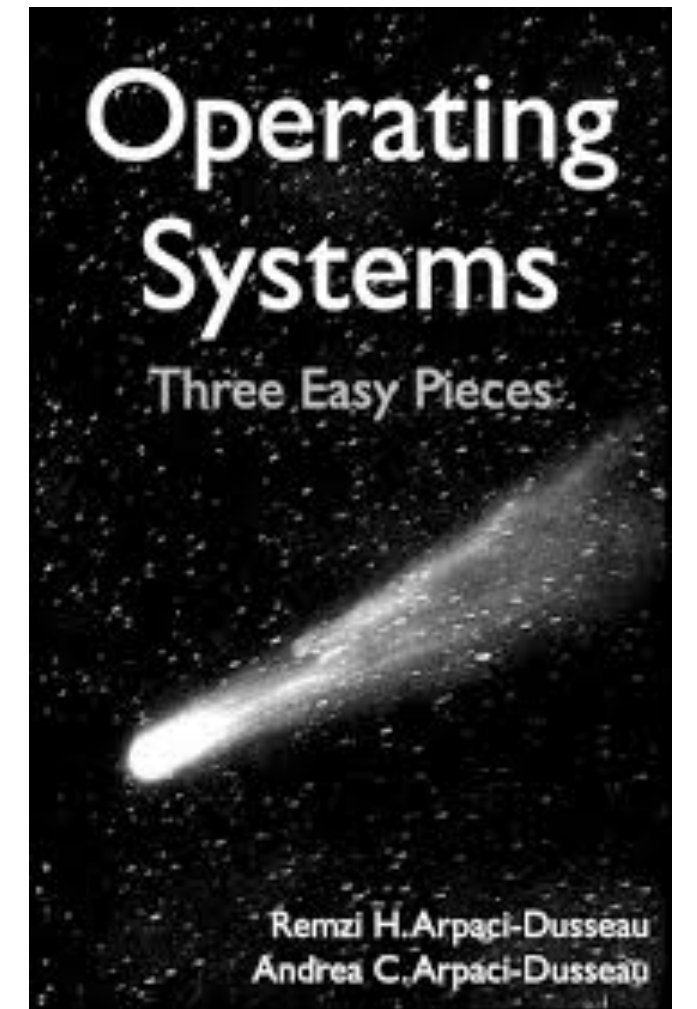
Assessment

- Paper Review (40%) – minimum grade: 10 values
 - Paper bidding: 27/02/2026
 - Initial reviews: 24/04/2026
 - Online paper discussion and review tuning: 04/05/2026
 - In-person paper review discussion: 08/05/2026

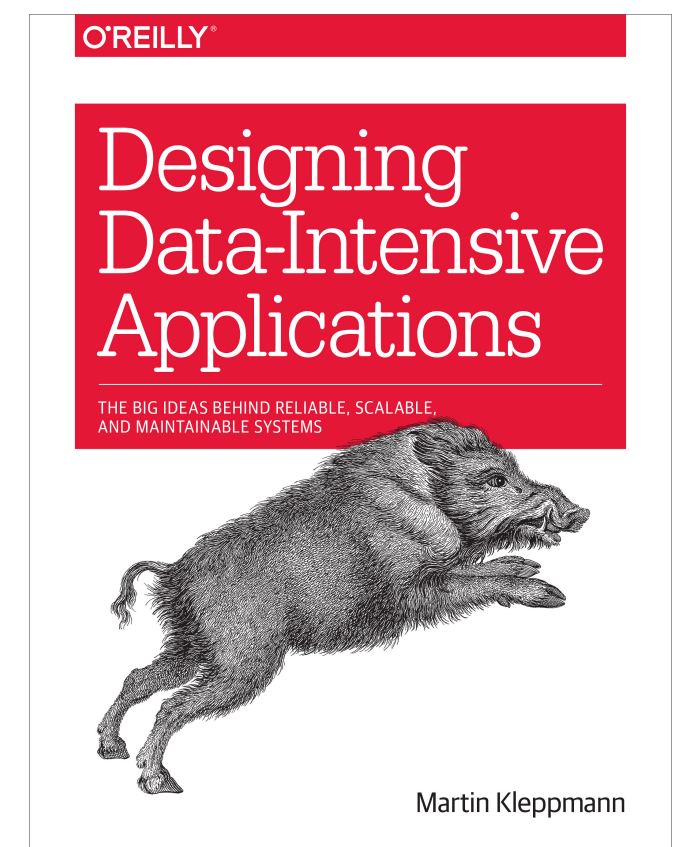
- Written exam (60%) – minimum grade: 8 values
 - 22/05/2026

Main References

- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau.
Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018.



- Martin Kleppman.
Designing Data-Intensive Applications: The big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly, 2017.



Team

© João Paulo - jtpaulo@di.uminho.pt



© Ricardo Macedo – d12010@di.uminho.pt



Questions?